프로젝트 계획서

네이버 API 기반 위치 기반 식사 추천 서비스 (Web-view App)

작성일: 2026. 03. 06 | 학번/이름: 3110 / 이동현 | ver 1.0

---

1. 프로젝트 개요

프로젝트명	위치 기반 식사 추천 서비스 (Web-view App)
학번 / 이름	3110 / 이동현
담당 역할	PM & Backend (프로젝트 관리, 서버 개발 및 인프라 구축)
개발 기간	2026. 03. 06 ~ 2026. 04. 25 (총 8주)

2. 기획 배경 및 목적

해결하고자 하는 문제

점심·저녁 식사 시 메뉴 선택에 소요되는 의사결정 비용(결정 장애)은 일상적으로 반복되지만 기존 서비스는 검색어 입력을 전제로 하여 능동적 탐색을 요구한다. 위치 자동 감지 기반의 수동적(passive) 추천 흐름이 필요하다.

기획 의도 및 목표

네이버 지역 검색 API를 프록시 서버로 경유하여 신뢰도 높은 식당 데이터를 수집하고, 터치 한 번으로 메뉴를 결정할 수 있는 최소 기능 제품(MVP)을 구축한다. 이후 사용자 취향 데이터 축적을 기반으로 개인화 추천 엔진으로 고도화하는 것을 장기 목표로 설정한다.

3. 서비스 상세 내용

타겟 사용자

점심·저녁 메뉴 결정에 어려움을 겪는 직장인 및 대학(원)생, 낯선 지역에서 식당 탐색이 필요한 여행자

핵심 기능 (Core Features)

기능	상세 설명
위치 기반 식당 탐색	HTML5 Geolocation API로 사용자 위경도 수집 후 Naver Reverse Geocoding으로 행정동 단위 주소 변환, 상단 UI 표시
메뉴 추천 엔진	현재 운영 중인 식당 중 카테고리별 무작위 큐레이션. 거리순(500m/1km/2km), 평점순, 카테고리(한/중/일/양식) 필터 제공
상세 정보 딥링크	식당 선택 시 네이버 플레이스 앱 또는 모바일 웹으로 직접 연결, 현재 위치 기준 길 찾기 연동

차별점

기존 서비스 (네이버 지도, 카카오맵 등)

- 검색어 입력 또는 카테고리 탭 선택 필요
- 방대한 정보로 인한 선택 피로도 증가
- 능동적 탐색 전제 UX

본 서비스 (차별화 포인트)

- 입력 없이 위치 감지만으로 즉시 추천
- 단일 큐레이션 결과 제시로 결정 비용 최소화
- 수동적(passive) 추천 UX로 사용자 이탈 방지

4. 기술 스택 (Tech Stack)

Frontend	Next.js (React) · TypeScript · Tailwind CSS SEO 및 빠른 초기 로딩(SSR) 대응 (팀원 담당)
Backend ★ 본인 담당	**Node.js** · **Express** · **Redis** · **dotenv (.env 보안)** 네이버 API 요청 대행 프록시 서버 구축, 캐싱 전략 설계, API 키 보안 관리 전담
App Shell	Flutter WebView · React Native WebView 네이티브 GPS 권한 및 딥링크 처리 (팀원 협업)
External API	Naver Search API (Local) · Naver Maps API · Naver Reverse Geocoding
DevOps / Infra ★ 본인 담당	**AWS EC2** · **GitHub Actions** · **Docker** · Redis (캐싱) CI/CD 파이프라인, Docker 컨테이너화, EC2 배포 환경 구축 전담
협업 툴 (PM)	**Notion** · **GitHub** · Figma · Discord Notion 일정 관리, GitHub Issues/Milestones 기반 진척 관리

5. 팀원 역할 및 책임 (R&R)

이름	담당 분야 (Role)	세부 업무 내용
조성현	Frontend (UI/UX)	Figma 화면 설계, 공통 컴포넌트 개발, 추천 화면 UI 구현, 반응형 웹 적용, 디자인 시스템 수립
김시우	Frontend (기능/연동)	HTML5 Geolocation 구현, Naver API 프론트 연동, 클라이언트 상태 관리, 필터링 로직, 딥링크 처리 및 WebView Bridge 연동
이동현 ★	**PM & Backend**	**프로젝트 일정 관리(Notion/GitHub), 네이버 API 프록시 서버 개발, Redis 캐싱 전략 설계, AWS EC2 배포 및 GitHub Actions CI/CD 구축**

6. 개발 일정 (WBS) — 이동현 담당 업무 기준

전체 기간: 2026. 03. 06 (금) ~ 2026. 04. 25 (금) / 8주

1~2주차	**기획 및 설계 / PM 착수** (03/06 ~ 03/19) — 요구사항 정의 및 팀 일정 계획 수립, Notion 프로젝트 보드 구성, 네이버 Cloud API 등록 및 키 발급, ERD 작성, API 명세서 초안 작성
3~4주차	**서버 기초 개발** (03/20 ~ 04/02) — Docker 개발 환경 구성, Node.js + Express 프록시 서버 초기화, Naver Search API / Reverse Geocoding Wrapper 모듈 완성, .env 기반 API 키 보안 설정
5~6주차	**서버 심화 개발** (04/03 ~ 04/16) — Redis 캐싱 레이어 적용(TTL 전략 수립), 필터링 파라미터 서버 가공 로직 개발, CORS 및 Rate Limit 미들웨어 설정, 프론트팀 연동 지원 및 API 응답 포맷 통일
7주차	**테스트 및 디버깅** (04/17 ~ 04/21) — 서버 부하 테스트, API 예외 처리 및 에러 코드 표준화, Redis 캐시 히트율 점검, 프론트팀 QA 이슈 백엔드 측 수정 지원
8주차	**배포 및 마무리** (04/22 ~ 04/25) — AWS EC2 프로덕션 배포, GitHub Actions CI/CD 파이프라인 완성, Docker 이미지 최적화, 서버 모니터링 설정, 최종 보고서 백엔드/인프라 파트 작성

7. 최종 산출물 목록 (Deliverables)

구분	항목
문서 산출물	요구사항 명세서, Figma 화면 설계서, DB 설계도(ERD), API 명세서
개발 산출물	소스코드 (GitHub Repository), 배포된 서비스 접속 URL, AOS/iOS WebView 빌드 파일
발표 산출물	최종 결과 보고서, 시연 영상 (선택)

8. 기대 효과 및 발전 방향

기대 효과

- **개발 효율성:** 네이버의 검증된 DB를 활용하여 식당 정보 수집·갱신 비용 절감, 데이터 신뢰성 확보
- **보안 안정성:** 프록시 서버 구조로 API 키 클라이언트 노출 차단, Redis 인메모리 캐싱으로 일일 API 쿼터 절약
- **사용자 편의성:** 검색어 입력 없이 터치 한 번으로 식사 메뉴 결정 프로세스를 단축하여 결정 피로도 해소

추후 발전 방향

- 사용자의 선택 이력 및 평가 데이터를 축적하여 개인화 AI 추천 모델로 고도화
- 동행인 수 및 예산 입력 기반의 그룹 추천 기능 추가
- 앱스토어(Google Play / App Store) 정식 배포 및 푸시 알림 연동

9. 기술적 고려사항 및 해결 전략

구분	리스크	해결 전략
API 보안	Client ID / Secret 클라이언트 노출로 인한 키 탈취 위험	서버사이드 Environment Variable (.env)로 키 관리, 클라이언트에서 직접 Naver API 호출 구조적 차단
API 쿼터	동일 지역 반복 요청으로 일일 호출 한도 조기 소진	Redis 캐싱 적용, 지역·카테고리 조합 키 기반 TTL 설정으로 데이터 신선도와 쿼터 효율 균형 유지
서버 가용성	EC2 단일 인스턴스 장애 시 서비스 중단	PM2 프로세스 매니저로 서버 자동 재시작, GitHub Actions 배포 실패 시 이전 버전 자동 롤백 설정
CI/CD 안정성	잦은 푸시로 인한 불안정한 배포	GitHub Actions에 lint·build 단계 필수화, main 브랜치 PR merge 시에만 자동 배포 트리거 설정